

# A cybersteps project

**Date:** 28. March 2026

**Participants:** Gregor

## Project: LearningSteps Evolution

### Executive summary:

This report documents the end-to-end deployment of a Python FastAPI application to Microsoft Azure using industry-standard DevOps tooling. The deployment covers containerization, cloud infrastructure provisioning, Kubernetes orchestration, and full CI/CD automation.

### Application

The deployed application is **LearningSteps API**, a FastAPI-based REST service with the following characteristics:

- **Framework:** FastAPI running on Uvicorn at port 8000
- **Database:** PostgreSQL (via SQLAlchemy + asyncpg)
- **Observability:** Prometheus metrics exposed at `/metrics`, structured console logging
- **Health check:** `/health` endpoint returning application status
- **Routing:** Modular router structure ( `journal_router` )

### Step 1 — Containerization with Docker

The application was packaged into a Docker container using a custom `Dockerfile`.

#### Key decisions:

- **Base image:** `python:3.11-slim` — lightweight, minimal attack surface
- **System dependencies:** `gcc` and `libpq-dev` installed to compile `psycpg2-binary` on the slim image
- **Layer caching:** `requirements.txt` is copied and installed before application code, so Docker only reinstalls dependencies when that file changes
- **Non-root user:** A dedicated `appuser` is created and used to run the application — the container never runs as root
- **Security context:** `readOnlyRootFilesystem` and `allowPrivilegeEscalation: false` enforced at the Kubernetes level

A `.dockerignore` file was added to exclude `__pycache__`, `.env`, `.git`, and other non-essential files from the image build context.

## Step 2 — Infrastructure Provisioning with Terraform

All cloud infrastructure was defined as code using Terraform ( `azurerm` provider `~> 3.0` ) and applied to Azure in the `westeurope` region. The configuration is split across three files:

| File                      | Purpose                       |
|---------------------------|-------------------------------|
| <code>main.tf</code>      | Resource definitions          |
| <code>variables.tf</code> | Input variables with defaults |
| <code>outputs.tf</code>   | Useful values after apply     |

**Resources provisioned:**

### Azure Resource Group

A single resource group ( `learningsteps-rg` ) contains all project resources, making cleanup straightforward.

### Azure Container Registry (ACR)

- SKU: `Basic` (sufficient for a single-project registry)
- Admin access disabled — authentication uses Azure RBAC exclusively
- The AKS cluster is granted the `AcrPull` role via a `azurerm_role_assignment`, so no credentials are required to pull images

### Azure Kubernetes Service (AKS)

- Single node pool, 1 node, `Standard_B2s` VM size
- `SystemAssigned` managed identity — no service principal secrets to manage or rotate
- Network plugin: `kubenet` with **Calico network policy** for pod-level firewall rules
- API server access restricted to authorized IP ranges

### Azure Database for PostgreSQL Flexible Server

- Version: PostgreSQL 15
- SKU: `B_Standard_B1ms` (burstable, cheapest tier — suitable for dev/test)
- Storage: 32 GB
- **Private DNS zone** ( `learningsteps.postgres.database.azure.com` ) linked to the AKS VNet — the database has no public endpoint and is only reachable from within the Azure

virtual network

- Application database ( `learningstepsdb` ) created as a separate Terraform resource
- Admin password marked `sensitive = true` — never committed to source control, passed via `TF_VAR_postgres_admin_password`

### Apply workflow:

bash

```
terraform init
terraform plan
terraform apply
```

## Step 3 — Kubernetes Deployment

After infrastructure was provisioned, `kubectl` was configured to target the AKS cluster:

bash

```
az aks get-credentials --resource-group learningsteps-rg --name
learningsteps-aks
```

Three Kubernetes manifests were applied:

### Secret ( `secret.yaml` )

Stores the `DATABASE_URL` connection string as a Kubernetes `Opaque` secret. The value is injected into the container as an environment variable at runtime — never baked into the Docker image.

### Deployment ( `deployment.yaml` )

- 1 replica of the application container
- Image pulled from ACR ( `learningstepsacr.azurecr.io/learningsteps:latest` )
- `DATABASE_URL` injected from the Kubernetes secret
- **Liveness probe:** `GET /health` — Kubernetes restarts the pod if this fails
- **Readiness probe:** `GET /health` — Kubernetes stops routing traffic to the pod until this passes
- **Prometheus annotations:** `prometheus.io/scrape` , `prometheus.io/path` , `prometheus.io/port` — enables automatic metric scraping
- **Resource limits:** CPU 500m / Memory 256Mi; requests CPU 100m / Memory 128Mi
- **Pod security context:** runs as UID 1000, non-root, read-only root filesystem

## Service ( `service.yaml` )

- Type: `LoadBalancer` — Azure automatically provisions a public IP
- External port 80 maps to container port 8000
- The application is publicly reachable at `http://<EXTERNAL-IP>/`

## Step 4 — CI/CD Automation with GitHub Actions

A GitHub Actions workflow ( `.github/workflows/deploy.yaml` ) automates the full build and deploy pipeline on every push to `main`.

**Authentication:** A service principal ( `learningsteps-github-actions` ) was created with `Contributor` scope on the resource group and stored as the `AZURE_CREDENTIALS` GitHub secret. The PostgreSQL password is stored separately as `POSTGRES_PASSWORD`.

### Pipeline jobs:

#### Job 1 — `build-and-push`

1. Checks out the repository
2. Logs in to Azure using `azure/login@v4`
3. Authenticates to ACR via `az acr login`
4. Tags the Docker image with the Git commit SHA ( `github.sha` ) for full traceability
5. Builds and pushes the image to ACR

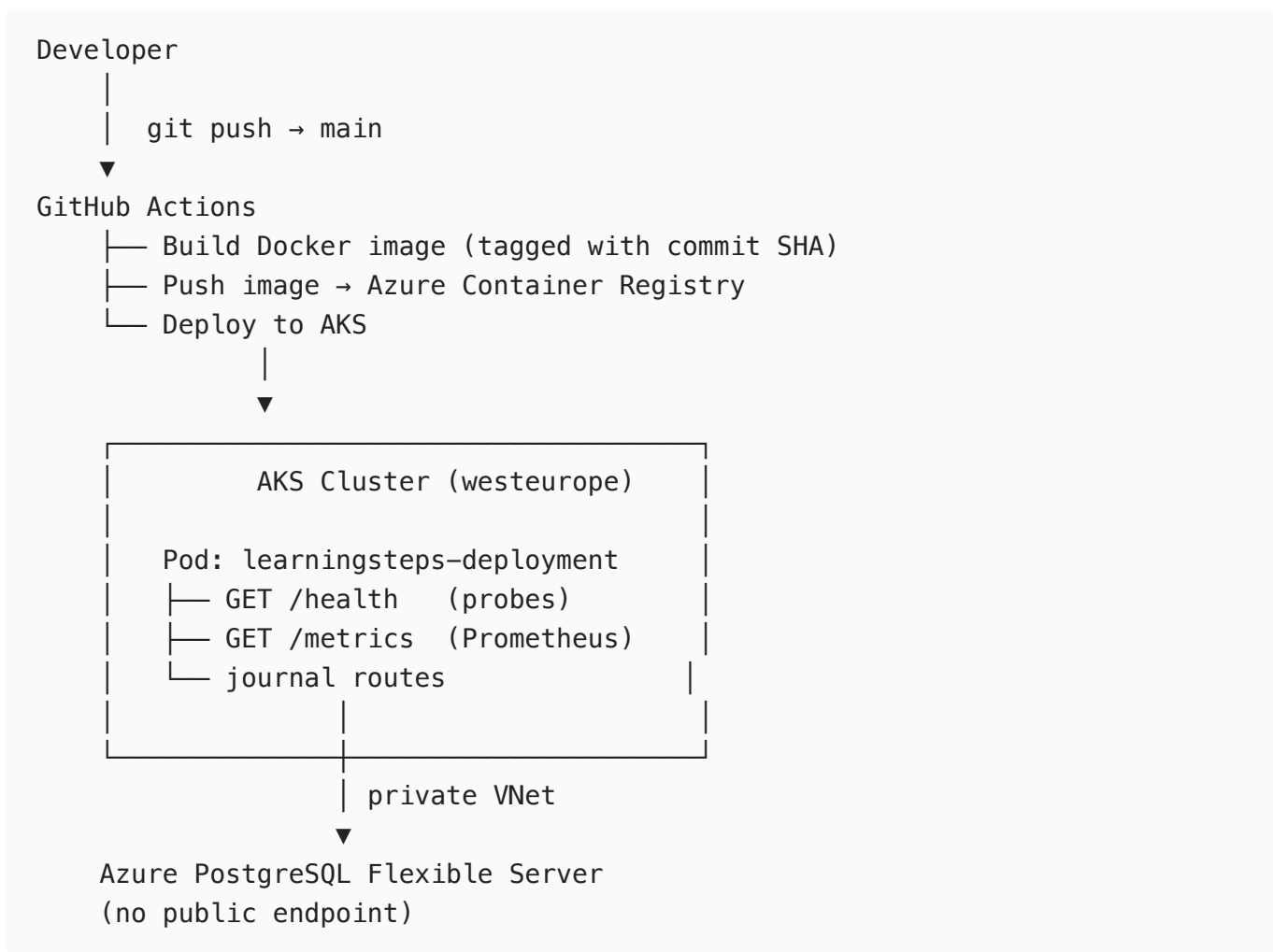
#### Job 2 — `deploy`

1. Retrieves AKS credentials via `az aks get-credentials`
2. Recreates the Kubernetes secret with the DB password from GitHub Secrets ( `--dry-run=client | kubectl apply` pattern for idempotency)
3. Updates the running deployment to the new image tag via `kubectl set image`
4. Waits for the rollout to complete ( `kubectl rollout status --timeout=120s` ) — the pipeline fails if pods don't become healthy within 2 minutes
5. Prints pod and service status as confirmation

Using the commit SHA as the image tag ensures that every deployment is uniquely identified and can be rolled back to any previous commit.

---

## Architecture Overview



## Security Highlights

| Area      | Measure                                                           |
|-----------|-------------------------------------------------------------------|
| Container | Non-root user, read-only filesystem, no privilege escalation      |
| Registry  | Admin credentials disabled, RBAC-only access                      |
| Database  | Private DNS zone, no public endpoint                              |
| Secrets   | Kubernetes Secrets + GitHub Actions Secrets, never in source code |
| Network   | Calico network policy, AKS API server IP restrictions             |
| Identity  | SystemAssigned managed identity, no long-lived credentials        |
| CI/CD     | Scoped service principal, commit-SHA image tags for traceability  |

## Tools & Versions

| Tool                  | Version       |
|-----------------------|---------------|
| Python                | 3.11          |
| FastAPI               | latest        |
| Terraform             | ≥ 1.3.0       |
| azurerm provider      | ~> 3.0        |
| PostgreSQL            | 15            |
| Kubernetes            | AKS-managed   |
| GitHub Actions runner | ubuntu-latest |